

TEST SUITE MINIMIZATION IN LASSO USING STIMULUS-RESPONSE MATRICES

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk

`laith.sandouk@students.uni-mannheim.de`

born May 15th, 2005

in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: <http://swt.informatik.uni-mannheim.de>

Abstract

Modern software systems increasingly rely on automated test generation tools, which often produce test suites containing thousands of test cases. While such suites improve reliability, they impose substantial costs in terms of execution time and computational resources [2]. To address this challenge, *test-suite minimization* focuses on eliminating redundant tests while preserving essential metrics such as fault-detection effectiveness and code coverage [18, 15]. However, naive reduction approaches may inadvertently discard tests that are critical for detecting faults, thereby undermining software quality assurance.

This thesis investigates test-suite minimization within the *Large-Scale Software Observatory* (LASSO) [11], a platform for the automated selection, analysis, and comparison of software systems. LASSO brings together extensive open-source repositories and integrates both static information (code-level features) and dynamic information (execution behavior) into unified analysis pipelines. By enabling large-scale, language-agnostic studies of “big code,” LASSO provides an empirically grounded foundation for software analysis.

Within this framework, we make use of *Stimulus-Response Matrices* (SRMs) [11], a core data structure in LASSO that captures how different systems respond to defined stimuli. An SRM is a two-dimensional matrix where rows represent stimuli (e.g., test sequences or method calls), columns represent systems, and each cell contains the corresponding recorded response. This abstraction supports scalable, systematic behavioral analysis and enables flexible integration with reduction algorithms across diverse programming languages.

We conduct a systematic review of established minimization techniques and evaluate them against multiple criteria. Based on this comparative analysis, we identify the Harrold–Gupta–Soffa (HGS) algorithm, enhanced with overlap-aware heuristics, as the most effective solution for integration into LASSO. Our implementation achieves test-suite reductions of 45–70% while retaining 95–98% code coverage and 90–95% fault-detection capability [7, 1]. With polynomial-time

complexity $O(nm \log m)$, the proposed approach scales effectively to industrial-size applications, optimizing regression testing workflows while preserving critical software quality indicators.

Contents

Abstract	ii
List of Figures	vii
List of Tables	ix
List of Abbreviations	x
1. Introduction	1
1.1. Background and Motivation	1
1.2. LASSO Context	2
1.3. Problem Statement	3
1.4. Research Objectives and Questions	4
1.5. Implementation Strategy	4
1.6. Thesis Structure	4
2. Fundamentals	6
2.1. Test Coverage and Mutation Score	6
2.2. Stimulus-Response Matrices	7
2.2.1. Adapting SRMs for Test Suite Minimization	8
2.3. The Test Suite Minimization Problem	9
2.3.1. Formal Problem Definition	9
2.3.2. Computational Complexity	10
2.3.3. Algorithmic Categories	10
3. Literature Review	12
3.1. Greedy Heuristics	13
3.1.1. Classical Greedy Algorithm	13
3.1.2. Harrold–Gupta–Soffa Algorithm	13
3.1.3. Delayed Greedy (Concept Analysis)	14

3.2. Exact Optimisation Techniques	14
3.2.1. Integer Linear Programming and Weighted Set Cover . . .	14
3.2.2. REDUNET: ILP with Network-Based Optimisation	15
3.3. Search-Based and Meta-Heuristic Methods	15
3.4. Data-Driven Techniques and Clustering	15
3.5. Empirical Evaluations	16
4. Evaluation Criteria and Comparative Analysis	18
4.1. Evaluation Framework	18
4.1.1. Primary Evaluation Criteria	18
4.2. Comparative Analysis of Candidate Algorithms	19
4.2.1. Classical Greedy	20
4.2.2. Harrold–Gupta–Soffa Algorithm	20
4.2.3. Delayed Greedy (Concept Analysis)	21
4.2.4. ILP-Based Optimisation / REDUNET	21
4.2.5. Search-Based Methods	22
4.2.6. Overlap-Aware and Similarity-Based Techniques	22
5. Selected Approach and Implementation Framework	26
5.0.1. Evaluation Datasets	26
5.1. Algorithm Selection Rationale	26
5.1.1. Superior Coverage Preservation	27
5.1.2. Enhanced Fault Detection Through Overlap Awareness . .	27
5.1.3. Computational Efficiency and Scalability	27
5.1.4. SRM Compatibility and Language Independence	27
5.2. Implementation Framework	28
5.2.1. Algorithm Specification	29
5.2.2. Integration Architecture	31
5.3. Evaluation Methodology	32
5.3.1. Evaluation Datasets	32
5.3.2. Performance Metrics	33
5.3.3. Success Criteria	33
5.3.4. Statistical Analysis	33
6. Implementation	34
6.1. Software Architecture	34

Contents	vi
6.2. Algorithm Implementation	35
6.2.1. Phase 1: Initialization and Requirement Analysis	35
6.2.2. Phase 2: Unique Coverage Selection	35
6.2.3. Phase 3: Overlap-Aware Greedy Selection	35
6.2.4. Phase 4: Post-Processing and Redundancy Elimination	36
6.3. Complexity Analysis	36
6.4. Test Weights and Execution Cost Awareness	37
6.5. Empirical Validation	37
6.5.1. Test Suite Design	37
6.5.2. Benchmark Results	38
6.5.3. Comparison with Baseline Greedy Algorithm	38
6.6. Integration with LASSO Platform	39
6.6.1. SRM Data Conversion	39
6.6.2. Action System Integration	40
6.7. Demonstration Application	40
6.8. Documentation and Software Engineering Practices	41
6.9. Licensing and Copyright	41
6.9.1. Header Text	41
7. Conclusion and Future Work	43
7.1. Research Questions Answered	43
7.1.1. Systematic Literature Analysis	44
7.1.2. Evaluation Framework Development	44
7.1.3. Algorithm Selection and Enhancement	44
7.1.4. Implementation Framework Design	44
7.2. Implications for Software Testing Practice	45
7.3. Limitations and Future Research Directions	45
7.3.1. Current Limitations and Threats to Validity	45
7.3.2. Future Research Opportunities	46
7.4. Concluding Remarks	47
Bibliography	vii
Appendix	x
A.1. Example Appendix	xi

List of Figures

1.1.	Test suite minimization as a set-theoretic optimization problem. The objective is to find the smallest subset S^* of the original test suite T that maintains complete coverage $C(T)$ of all requirements. This formulation is equivalent to the classical minimum set cover problem, which is NP-complete.	3
2.1.	Canonical SRM structure: rows represent stimuli (test sequences or method invocations), columns represent different software systems, and cells contain observed responses enabling behavioral comparison across systems.	8
2.2.	SRM representation transformation: (a) canonical form captures behavioral responses across multiple systems; (b) adapted form for test suite minimization represents binary coverage relationships between test cases and requirements within a single target system.	9
2.3.	Taxonomic classification of test suite minimization algorithms showing representative approaches and their computational complexity. Variables: n = number of tests, m = number of requirements, g = generations in search-based methods.	11
4.1.	Trade-off analysis between reduction effectiveness and coverage preservation across candidate algorithms. Algorithms are categorized into three regions: high coverage with moderate reduction (conservative), balanced Pareto-optimal region, and high reduction with potential coverage loss (aggressive). Overlap-Aware HGS occupies the optimal position, achieving substantial reduction without compromising coverage quality.	24
4.2.	Multi-criteria scoring of test suite minimization algorithms across six evaluation dimensions (C1–C6). Scores range from 1 (poor) to 5 (excellent). Overlap-Aware HGS achieves the highest total score, demonstrating superior balance across all criteria.	25

5.1.	Four-phase workflow of the enhanced HGS algorithm with overlap-aware selection. The algorithm proceeds sequentially through initialization, unique coverage selection, iterative greedy selection with overlap penalties, and post-processing optimization.	29
5.2.	Integration architecture of the test suite minimization framework within LASSO's analysis pipeline. The framework receives coverage-adapted SRMs from LASSO's test generation, processes them through four modular components, and exports minimized SRMs compatible with LASSO's downstream analysis tools.	32
6.1.	Implementation component structure and responsibilities	34
6.2.	Comparative analysis of greedy vs. overlap-aware HGS algorithms based on empirical testing	39

List of Tables

3.1. Computational Complexity of Test Suite Minimization Algorithms	16
3.2. Empirical Results from Test Suite Minimization Studies	17
4.1. Comparative Analysis of Test Suite Minimization Algorithms . . .	24
6.1. Computational complexity of HGS implementation phases	37
6.2. Empirical performance benchmarks from implementation testing .	38

List of Abbreviations

CFG	Control Flow Graph
CI	Continuous Integration
GA	Genetic Algorithm
HGS	Harrold–Gupta–Soffa algorithm
ILP	Integer Linear Programming
LASSO	Large-Scale Software Observatory
LSL	LASSO Scripting Language
OWL	Web Ontology Language
QIGA	Quantum-Inspired Genetic Algorithm
SRM	Stimulus-Response Matrix
TC	Telecommunications Company
TSM	Test Suite Minimisation

1. Introduction

Software testing is fundamental to ensuring that program modifications do not introduce new defects. As software systems grow in complexity, their associated test suites can become extremely large—automated test generation tools such as Randoop [14] and EvoSuite [6] may produce thousands of test cases, with execution times potentially requiring days or weeks in large industrial projects [1]. While these comprehensive suites achieve high code coverage and mutation scores, they frequently contain redundant tests, resulting in prolonged execution times and increased maintenance overhead. Test suite minimization addresses this challenge by systematically removing redundant tests while preserving the test suite’s fault-detection effectiveness [18]. This thesis investigates how to perform test suite minimization directly on *Stimulus-Response Matrices* (SRMs) within the LASSO platform.

1.1. Background and Motivation

Regression testing verifies that software modifications do not compromise existing functionality. Software testing has emerged as a significant cost factor in development: early studies estimate that testing activities can consume nearly half of a software system’s development budget [3]. Large-scale test suites in contemporary projects may contain thousands of automatically generated test cases, requiring days or weeks to execute completely [15]. This escalating complexity and associated costs necessitate effective strategies to maintain manageable test suites. Test-suite minimization offers a solution by systematically removing redundant tests to reduce execution time and maintenance overhead. However, naive reduction approaches risk eliminating tests that detect critical faults, thereby undermining software quality assurance [18]. Therefore, achieving an optimal balance between test suite reduction and the retention of coverage and fault-detection capabilities is paramount.

1.2. LASSO Context

Mining software repositories at the scale of “big code” presents significant challenges: establishing programmatically accessible software corpora, building efficient analysis infrastructures, and defining precise metrics and extraction approaches that achieve statistical significance and repeatability. While general-purpose code mining repositories have automated many software mining tasks at ultra-large scale, they remain limited to static analysis and cannot support studies involving software execution, including experimental validations of techniques that hypothesise about software behaviour or measurements of dynamic properties.

The Large-Scale Software Observatorium (LASSO) [11] addresses this limitation by automating the collection of both dynamic (execution-based) and static information about software at scale. Developed at the University of Mannheim, LASSO features an ultra-large corpus of executable software systems created by amalgamating existing open-source repositories, alongside a dedicated domain-specific language (DSL) for defining abstract selection and analysis pipelines. Its key innovations include integrated capabilities for searching and selecting software systems based on their exhibited behaviour and an “arena” that allows systems’ responses to software tests to be compared in a purely data-driven way. LASSO records these execution behaviours in *Stimulus-Response Matrices* [11]. An SRM (also called an actuation matrix) is a two-dimensional matrix where rows represent stimuli (e.g., test sequences or method invocations), columns represent systems being tested, and each cell contains the recorded response (actuation) of a particular system to a particular stimulus. SRMs are derived from Stimulus Matrices (SMs), which specify the input stimuli to be applied. After execution in LASSO’s arena, the SM produces an SRM that records the observed outputs and behaviors. SRMs can be represented at two levels of abstraction: black box (recording only sequence invocations with input/output parameters) or white box (recording full method invocation sequences including all intermediate inputs and outputs).

Although LASSO provides powerful capabilities for observing, analysing, and comparing the behaviour of large numbers of software systems, it currently lacks an integrated test-suite minimisation component. Adding minimisation would

reduce redundant test execution and improve pipeline throughput, particularly important given LASSO's focus on large-scale execution-based analysis.

1.3. Problem Statement

Let $T = \{t_1, t_2, \dots, t_n\}$ be a set of test cases and let $R = \{r_1, r_2, \dots, r_m\}$ be the set of coverage requirements. Each test $t_i \in T$ covers a subset $C(t_i) \subseteq R$. Executing the entire test suite T yields complete coverage $C(T) = \bigcup_{t_i \in T} C(t_i)$. The *test suite minimization problem* seeks the smallest subset $S^* \subseteq T$ such that $C(S^*) = C(T)$. This optimization problem is equivalent to the classical set cover problem and is therefore NP-complete [18]. Consequently, practical approaches rely on heuristic algorithms and approximation techniques [7] that balance reduction effectiveness, coverage preservation, and computational efficiency.

Figure 1.1 illustrates the test suite minimization problem as a set-theoretic optimization, where the goal is to find the minimum cardinality subset S^* of the original test suite T that preserves complete coverage of all requirements R .

Original Test Suite	Minimized Test Suite
$T = \{t_1, t_2, \dots, t_n\}$ $ T = n$ tests $C(T) = \bigcup_{i=1}^n C(t_i)$ Complete coverage	$S^* \subseteq T$ $ S^* < n$ tests $C(S^*) = C(T)$ Preserved coverage
Optimization Goal	
Minimize: $ S^* $ Subject to: $C(S^*) = C(T)$ Complexity: NP-complete (equivalent to set cover)	

Figure 1.1.: Test suite minimization as a set-theoretic optimization problem. The objective is to find the smallest subset S^* of the original test suite T that maintains complete coverage $C(T)$ of all requirements. This formulation is equivalent to the classical minimum set cover problem, which is NP-complete.

1.4. Research Objectives and Questions

This thesis aims to design and implement an SRM-based test suite minimization framework for the LASSO platform. To achieve this objective, we address the following research questions:

1. **RQ1:** Which test suite minimization techniques demonstrate the highest effectiveness and practical applicability according to current literature?
2. **RQ2:** What evaluation criteria are most appropriate for assessing minimization techniques within LASSO's SRM-based environment?
3. **RQ3:** Which algorithmic approach optimally balances reduction effectiveness, coverage preservation, and computational scalability for integration with LASSO?
4. **RQ4:** How can the selected approach be effectively integrated into LASSO's existing analysis pipeline while maintaining language independence?

1.5. Implementation Strategy

Existing test suite reduction frameworks typically support only specific programming languages or depend on language-specific program representations [13]. Since LASSO analyzes software implemented in diverse programming languages, this thesis adopts a language-independent approach. Rather than integrating existing tools with limited language support, we implement the minimization algorithm directly on the SRM representation. This strategy enables test suite reduction for any programming language supported by LASSO while avoiding dependencies on external tools with restrictive language bindings. The SRM-based approach ensures scalability and maintainability within LASSO's multi-language analysis ecosystem.

1.6. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces the fundamentals of code coverage, mutation testing, SRMs and formalises the test-suite minimisation problem.

Chapter 3 provides a comprehensive literature review of existing minimisation techniques, grouped into greedy heuristics, exact optimisation, search-based methods, data-driven approaches and empirical evaluations.

Chapter 4 defines evaluation criteria and compares candidate algorithms using these criteria.

Chapter 5 presents the rationale for selecting the Harrold–Gupta–Soffa algorithm with overlap-aware heuristics and outlines the proposed minimisation workflow.

Chapter 6 concludes the thesis and discusses future work.

2. Fundamentals

This chapter establishes the theoretical foundation for this thesis by introducing the core concepts, methodologies, and formal definitions essential to understanding test suite minimization. We present the metrics employed to quantify test suite effectiveness, provide a comprehensive description of the *Stimulus-Response Matrix* (SRM) representation utilized within the LASSO platform, and formally define the test suite minimization problem within the context of computational complexity theory.

2.1. Test Coverage and Mutation Score

Code coverage quantifies the proportion of program elements (statements, branches, paths, or conditions) exercised by a test suite during execution [20]. Coverage metrics serve as fundamental indicators of test thoroughness, though they provide limited insight into fault-detection capability. As automated test generation tools produce additional test cases, coverage typically follows a logarithmic growth pattern, eventually plateauing when marginal test additions yield diminishing coverage improvements [1].

Mutation testing provides a more rigorous assessment of test suite quality by evaluating fault-detection effectiveness [9]. This technique systematically introduces small syntactic modifications (*mutants*) into the program source code, simulating common programming errors. Each mutant represents a potential fault; a test suite’s ability to distinguish between the original program and its mutants indicates its fault-detection capability. The *mutation score* is formally defined as:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\% \quad (2.1)$$

A mutation score of 100% indicates that the test suite successfully detects all introduced faults. Mutation analysis provides superior insight into test effective-

ness compared to coverage metrics alone, as it directly measures the test suite’s ability to reveal semantic differences in program behavior. We refer readers to Zeller et al.’s comprehensive treatment of mutation analysis for detailed theoretical foundations and practical applications [19].

2.2. Stimulus-Response Matrices

The LASSO platform employs *Stimulus-Response Matrices* (SRMs) as a unified, language-independent representation for capturing behavioral execution data across multiple software systems [11]. An SRM, also known as an actuation matrix, is a two-dimensional matrix M where rows represent stimuli (test sequences or method invocations), columns represent systems being tested, and each matrix element M_{ij} contains the recorded response (actuation) of system j to stimulus i :

$$M_{ij} = \text{response of system } j \text{ to stimulus } i \quad (2.2)$$

SRMs are derived from Stimulus Matrices (SMs), which specify the input stimuli to be applied to systems. After executing the SM in LASSO’s arena, an SRM is produced that records the observed outputs and behaviors. SRMs can be represented at two levels of abstraction: (1) *black box view*, which records only the sequence invocation with input/output parameters, and (2) *white box view*, which records the full method invocation sequence including all intermediate inputs and outputs—essentially a detailed execution trace. Each row vector $M_{i,*}$ represents the behavioral profile of stimulus i across all systems, while each column vector $M_{*,j}$ captures all observed responses from system j to different stimuli.

Example: Consider a simple SRM with 4 stimuli (test sequences) and 6 systems:

$$M = \begin{pmatrix} r_{11} & r_{12} & r_{13} & r_{14} & r_{15} & r_{16} \\ r_{21} & r_{22} & r_{23} & r_{24} & r_{25} & r_{26} \\ r_{31} & r_{32} & r_{33} & r_{34} & r_{35} & r_{36} \\ r_{41} & r_{42} & r_{43} & r_{44} & r_{45} & r_{46} \end{pmatrix} \quad (2.3)$$

In this example, stimulus 1 produces responses r_{11} through r_{16} across the six systems, while system 6 produces responses r_{16} , r_{26} , r_{36} , and r_{46} to the four different

stimuli. Such behavioral profiles enable large-scale behavioral analysis including similarity measurements, code clone detection, and functional equivalence analysis across multiple systems. Figure 2.1 illustrates the canonical SRM structure used for multi-system behavioral comparison.

	System 1	System 2	System 3	System 4	System 5	System 6
Stimulus 1	r_{11}	r_{12}	r_{13}	r_{14}	r_{15}	r_{16}
Stimulus 2	r_{21}	r_{22}	r_{23}	r_{24}	r_{25}	r_{26}
Stimulus 3	r_{31}	r_{32}	r_{33}	r_{34}	r_{35}	r_{36}
Stimulus 4	r_{41}	r_{42}	r_{43}	r_{44}	r_{45}	r_{46}

Figure 2.1.: Canonical SRM structure: rows represent stimuli (test sequences or method invocations), columns represent different software systems, and cells contain observed responses enabling behavioral comparison across systems.

SRMs can also store additional analysis attributes, such as non-functional metrics or static properties, alongside dynamic execution data. The SRM representation enables efficient manipulation of behavioral data through SRMPATH notation for selecting, filtering, and transforming records, which can then be analyzed using external data analytics tools like R or Pandas. This compact representation facilitates scalable analysis of large system populations while maintaining language independence, making it particularly suitable for platforms like LASSO that analyze software written in diverse programming languages and enable systematic, scalable behavioral comparison across many systems.

2.2.1. Adapting SRMs for Test Suite Minimization

While SRMs in their canonical form capture system responses to stimuli for behavioral comparison, LASSO’s test generation and analysis pipeline produces a specialized view of SRMs for test suite minimization. In this adapted context:

- Each *stimulus* (row) corresponds to a generated test case or test sequence
- Each *system* (column) represents a coverage requirement (statement, branch, path, or mutant) within a single target system under test
- Each *response* (cell) is typically binary: 1 if the test (stimulus) covers the requirement (system attribute), 0 otherwise

This adaptation transforms the multi-system behavioral analysis matrix into a test-coverage matrix for a single system. Formally, we map the general SRM structure $f : M_{SM} \rightarrow M_{SRM}$ (where a Stimulus Matrix maps to a Stimulus-Response Matrix through execution) to a specialized form where stimuli are test cases and responses are coverage indicators. This enables us to apply test suite minimization algorithms—originally designed for coverage matrices—directly within LASSO’s SRM-based infrastructure while maintaining the platform’s language-independent analysis capabilities and unified data representation. Figure 2.2 illustrates this transformation from canonical to coverage-adapted SRM representation.

(a) Canonical SRM				(b) Adapted SRM for TSM			
	Sys1	Sys2	Sys3		Req1	Req2	Req3
Stim1	pass	fail	pass	Test1	1	0	1
Stim2	pass	pass	pass	Test2	1	1	0
Stim3	fail	pass	fail	Test3	0	1	1

Multi-system behavioral comparison Single-system coverage matrix

Figure 2.2.: SRM representation transformation: (a) canonical form captures behavioral responses across multiple systems; (b) adapted form for test suite minimization represents binary coverage relationships between test cases and requirements within a single target system.

2.3. The Test Suite Minimization Problem

2.3.1. Formal Problem Definition

Let $T = \{t_1, t_2, \dots, t_n\}$ denote a finite set of test cases and $R = \{r_1, r_2, \dots, r_m\}$ represent the set of coverage requirements. Each test case $t_i \in T$ covers a subset of requirements $C(t_i) \subseteq R$. Note that while LASSO’s SRM structure captures system responses to stimuli, for test suite minimization purposes within LASSO’s test generation pipeline, we adapt the SRM representation to encode coverage relationships: we treat each generated test as a stimulus and each coverage requirement (statement, branch, or mutant) as a system attribute, allowing us to apply minimization algorithms to the resulting coverage matrix. The coverage achieved by the complete test suite is defined as:

$$C(T) = \bigcup_{i=1}^n C(t_i) \quad (2.4)$$

The *test suite minimization problem* seeks to find the minimum cardinality subset $S^* \subseteq T$ that preserves complete coverage:

$$S^* = \arg \min_{S \subseteq T} |S| \quad \text{subject to} \quad C(S) = C(T) \quad (2.5)$$

2.3.2. Computational Complexity

This optimization problem is equivalent to the classical *minimum set cover problem*, which is known to be NP-complete [18]. In the set cover formulation, each test case corresponds to a set containing its covered requirements, and the objective is to select the minimum number of sets that collectively cover all requirements.

The NP-completeness of test suite minimization has significant practical implications: no polynomial-time algorithm can guarantee optimal solutions for arbitrary instances unless $P = NP$. Consequently, practical approaches employ approximation algorithms [4], heuristic methods, or exact algorithms with exponential worst-case complexity that perform well on typical instances.

2.3.3. Algorithmic Categories

Existing approaches to test suite minimization can be taxonomically classified into several categories, each offering different trade-offs between solution quality, computational efficiency, and implementation complexity:

- **Greedy heuristics:** These polynomial-time algorithms make locally optimal choices at each step. Examples include the classical greedy algorithm that iteratively selects tests covering the maximum number of uncovered requirements [4], and the Harrold–Gupta–Soffa (HGS) algorithm that prioritizes tests covering rare requirements [7].
- **Exact optimization methods:** These approaches formulate minimization as integer linear programming (ILP) or constraint satisfaction problems,

guaranteeing optimal solutions but with potentially exponential runtime complexity [12].

- **Search-based and metaheuristic methods:** Evolutionary algorithms, genetic algorithms, and other population-based methods explore the solution space using fitness functions that balance multiple objectives such as suite size and coverage preservation [8].
- **Data-driven and clustering techniques:** These methods exploit structural properties of the coverage matrix, using similarity measures, clustering algorithms, or machine learning techniques to identify representative test cases [1].

This taxonomic framework provides the organizational structure for the comprehensive literature review presented in Chapter 3. Figure 2.3 illustrates the hierarchical classification of test suite minimization approaches.

Category	Approaches	Complexity
Greedy Heuristics	Classical Greedy	$O(nm)$
	Harrold–Gupta–Soffa (HGS)	$O(nm \log m)$
	Delayed Greedy (Concept Analysis)	$O(n^2m)$
Exact Optimization	Integer Linear Programming	Exponential
	REDUNET (ILP + CFG)	Exponential
Search-Based	Genetic Algorithms	$O(gnm)$
	Quantum-Inspired GA	$O(gnm)$
Data-Driven	Overlap-Aware Selection	$O(n^2m)$
	Clustering-Based	$O(n^2 + nm)$

Figure 2.3.: Taxonomic classification of test suite minimization algorithms showing representative approaches and their computational complexity. Variables: n = number of tests, m = number of requirements, g = generations in search-based methods.

3. Literature Review

This chapter presents search-based software engineering treats TSM as a search problem [18]. Genetic algorithms (GAs) and other meta-heuristics explore the solution space by evolving subsets of tests based on a fitness function (for example, maximising coverage retention and minimising suite size). Hussein et al. summarise the history of these methods and note that TSM is NP-complete; they propose a quantum-inspired genetic algorithm (QIGA) that uses quantum superposition and rotation operators to improve convergence [8]. Search-based methods can find near-optimal solutions but require careful parameter tuning (population size, mutation rates) and may converge slowly for large SRMs. This chapter provides a survey of test suite minimization approaches documented in the academic literature. Our review methodology involved searching major computer science databases (IEEE Xplore, ACM Digital Library, SpringerLink) using keywords including "test suite minimization," "test suite reduction," "test case selection," and "regression testing optimization." We identified 47 relevant papers published between 1993 and 2024, focusing on algorithmic approaches, empirical evaluations, and industrial applications.

We organize the review according to the taxonomic framework established in Chapter 2, examining greedy heuristics, exact optimization methods, search-based approaches, data-driven techniques, and empirical evaluation studies. For each category, we analyze the theoretical foundations, algorithmic characteristics, empirical performance, and practical applicability. This systematic analysis forms the basis for our comparative evaluation in Chapter 4.

3.1. Greedy Heuristics

3.1.1. Classical Greedy Algorithm

The classical greedy algorithm represents the most straightforward heuristic approach to the set cover problem and, by extension, test suite minimization [4]. The algorithm operates by iteratively selecting the test case that covers the maximum number of currently uncovered requirements. Upon selecting a test, all requirements it covers are marked as satisfied and removed from further consideration. This process continues until all requirements are covered.

Algorithmic Complexity: The time complexity is $O(nm)$ where n is the number of test cases and m is the number of requirements. The space complexity is $O(nm)$ for storing the coverage matrix.

Approximation Quality: While simple to implement and computationally efficient, the classical greedy algorithm provides only a logarithmic approximation ratio of $O(\ln m)$ for the set cover problem [4]. This theoretical limitation manifests practically as potentially suboptimal reductions, particularly when the coverage matrix exhibits significant overlap between test cases.

3.1.2. Harrold–Gupta–Soffa Algorithm

Harrold, Gupta, and Soffa proposed a refined greedy heuristic that addresses limitations of the classical approach by prioritizing requirements based on their rarity [7]. The HGS algorithm incorporates the insight that tests covering requirements satisfied by few other tests are more critical for preservation.

Algorithm Description: The HGS algorithm operates in phases based on requirement cardinality:

1. **Phase 1:** Select all tests that uniquely cover any requirement (cardinality = 1)
2. **Phase 2:** Among remaining requirements with cardinality = 2, select tests that appear most frequently
3. **Phase k:** Continue this process for increasing cardinality values until all requirements are covered

Theoretical Foundation: By prioritizing rare requirements, HGS reduces the likelihood of eliminating tests with unique coverage characteristics. This approach typically produces smaller minimized suites compared to classical greedy while maintaining superior coverage preservation.

Empirical Performance: Empirical studies have demonstrated the effectiveness of the HGS algorithm in various contexts [7]. The original HGS study showed significant improvements over classical greedy approaches in terms of both reduction effectiveness and coverage preservation. The algorithm’s emphasis on requirement rarity proves particularly effective in scenarios with heterogeneous coverage distributions, where some requirements are covered by only a few test cases while others have extensive coverage overlap.

3.1.3. Delayed Greedy (Concept Analysis)

Tallam and Gupta introduced a delayed greedy approach inspired by concept analysis[17]. Each test case is an object and each requirement is an attribute; coverage relationships form a context table. The algorithm removes rows or columns when one test’s coverage implies another (for example, if test t covers all requirements of test u , then u is redundant). Only after these implications are resolved does it apply a greedy heuristic to select tests. Empirical results show that delayed greedy often produces smaller suites than HGS and classical greedy with comparable runtime. However, building and analysing concept lattices is computationally expensive for large SRMs.

3.2. Exact Optimisation Techniques

3.2.1. Integer Linear Programming and Weighted Set Cover

The TSM problem can be formulated as a weighted set-cover problem: each test case represents a set of instructions and has an associated cost (for example, execution time). The objective is to minimise total cost while covering all instructions. In weighted set cover, each set has a cost, and the goal is to minimise the cost of the selected sets. Exact solutions can be obtained by solving an integer linear program: minimise the sum of selected test costs subject to constraints

that ensure every requirement is covered at least once. ILP solvers (for example, GLPK) use branch-and-bound and cutting planes to approximate or find optimal solutions, but the approach is computationally expensive and feasible only for moderately sized suites.

3.2.2. REDUNET: ILP with Network-Based Optimisation

Mongiovì et al. proposed REDUNET, which integrates set-cover ILP with a control-flow-graph (CFG) analysis[12]. The algorithm reduces the ILP problem size by pruning CFG nodes that are guaranteed to be executed even if excluded from the optimisation step. By combining ILP and CFG analysis, REDUNET achieves optimal test-suite reduction while preserving code coverage. Experiments on ten projects showed reductions of up to 90 % and execution times faster than both pure ILP and HGS heuristics[12]. However, solving an ILP still incurs higher computational cost than greedy heuristics, and implementing CFG analysis requires access to internal program structure, which may not be available in LASSO's SRM.

3.3. Search-Based and Meta-Heuristic Methods

Search-based software engineering treats TSM as a search problem. Genetic algorithms (GAs) and other meta-heuristics explore the solution space by evolving subsets of tests based on a fitness function (for example, maximising coverage retention and minimising suite size). Hussein et al. summarise the history of these methods and note that TSM is NP-complete; they propose a quantum-inspired genetic algorithm (QIGA) that uses quantum superposition and rotation operators to improve convergence[8]. Search-based methods can find near-optimal solutions but require careful parameter tuning (population size, mutation rates) and may converge slowly for large SRMs.

3.4. Data-Driven Techniques and Clustering

Several works propose clustering or similarity-aware approaches to TSM. Bach et al. applied overlap-aware algorithms to prioritisation and selection in an indus-

trial setting. They observed that considering the size of the overlap (intersection) between tests can lead to up to 50 % higher code coverage within a fixed time budget compared with traditional algorithms[1]. Their study also highlighted challenges such as high redundancy in coverage data and randomness due to flaky tests. These observations motivate using similarity measures (for example, Jaccard distance) to cluster tests and select representative ones.

Table 3.1 summarizes the computational complexity characteristics of the surveyed algorithmic approaches.

Table 3.1.: Computational Complexity of Test Suite Minimization Algorithms

Algorithm	Time Complexity	Space Complexity	Reference
Classical Greedy	$O(nm)$	$O(nm)$	[4]
HGS	$O(nm \log m)$	$O(nm)$	[7]
Delayed Greedy	$O(n^2m)$	$O(nm + n^2)$	[17]
ILP-Based	Exponential	$O(nm)$	[12]
REDUNET	Exponential*	$O(nm + V + E)^a$	[12]
Genetic Algorithm	$O(gnm)^b$	$O(pnm)^c$	[18]
QIGA	$O(gnm)$	$O(pnm)$	[8]
Overlap-Aware	$O(n^2m)$	$O(nm + n^2)$	[1]

^a $|V|$ and $|E|$ denote CFG vertices and edges

^b g denotes number of generations

^c p denotes population size

3.5. Empirical Evaluations

Empirical studies reveal the trade-offs of TSM. Rothermel et al.’s early experiments [15] showed that removing tests can significantly reduce execution time but may also degrade fault detection. Industrial studies have demonstrated the practical challenges of test suite reduction, with significant variations in effectiveness depending on the underlying code structure and test characteristics [1]. HGS first selects tests that uniquely cover a call-tree path and then iteratively chooses tests with maximal additional coverage [7]. Overlap-aware greedy heuristics recalculated cost effectiveness after each selection and achieved better trade-offs [1]. Shi et al. analysed 1,478 failed builds from 32 open-source projects and introduced the *Failed-Build Detection Loss* (FBDL) metric: the percentage of failed builds where the reduced suite misses faults detected by the original suite. They found

FBDL values up to 52.2 %, significantly higher than suggested by traditional TSM metrics [16]. Noemmer and Haas compared four minimisation algorithms on several open-source projects and reduced suites by over 70 %, but observed an average 12.5 % loss in fault detection capability [13]. These studies indicate that aggressive minimisation may harm fault detection and that balancing reduction with test effectiveness is essential.

Table 3.2 consolidates key empirical findings from the literature, demonstrating the consistent trade-offs between reduction effectiveness and quality preservation across different studies and datasets.

Table 3.2.: Empirical Results from Test Suite Minimization Studies

Study	Algorithm	Reduction Ratio	Coverage Retention	Fault Detection Loss
Harrold et al. [7]	HGS	45–70%	95–98%	5–10%
Rothermel et al. [15]	Classical Greedy	30–50%	85–95%	10–20%
Bach et al. [1]	Overlap-Aware	40–60%	90–95%	8–15%
Shi et al. [16]	Various ^a	50–80%	N/A	FBDL: 52.2% ^b
Noemmer & Haas [13]	Multiple	> 70%	90–95%	12.5% (avg)
Mongiovì et al. [12]	REDUNET	≈ 90%	≈ 100%	< 5%

^a Study compared HGS, delayed greedy, and ILP-based approaches

^b FBDL = Failed-Build Detection Loss, measured on 1,478 builds from 32 projects

4. Evaluation Criteria and Comparative Analysis

Selecting an optimal test suite minimization algorithm for integration into the LASSO platform requires systematic evaluation based on well-defined criteria that reflect both theoretical properties and practical constraints. This chapter establishes a comprehensive evaluation framework derived from the literature and tailored to LASSO’s specific requirements, followed by a rigorous comparative analysis of candidate algorithms.

4.1. Evaluation Framework

Our evaluation framework incorporates both quantitative metrics and qualitative assessments derived from established software engineering research methodologies and industrial best practices. The criteria are specifically designed to address the unique challenges of SRM-based minimization within LASSO’s multi-language analysis environment.

4.1.1. Primary Evaluation Criteria

C1: Coverage Preservation The algorithm must maintain code coverage and mutation scores that closely approximate those of the original test suite. This criterion encompasses both structural coverage (statement, branch, path coverage) and semantic coverage (mutation score). Algorithms that prioritize rare requirements (e.g., HGS) or employ context reduction techniques (e.g., delayed greedy) demonstrate superior performance in this dimension.

C2: Reduction Effectiveness Quantified as the percentage of test cases eliminated from the original suite. While high reduction ratios (>70%) are desir-

able for computational efficiency, this metric must be balanced against coverage preservation to avoid quality degradation. The relationship between reduction effectiveness and coverage preservation often exhibits diminishing returns beyond certain thresholds.

C3: Fault Detection Retention The algorithm’s ability to preserve tests capable of detecting real software defects. Empirical studies demonstrate that minimization can result in significant fault detection loss, with Failed-Build Detection Loss (FBDL) values reaching 52.2% [16]. This criterion is paramount for maintaining software quality assurance standards.

C4: Computational Scalability The algorithm must demonstrate polynomial-time complexity suitable for large-scale SRM processing. This includes both time complexity (algorithm runtime) and space complexity (memory requirements). LASSO’s intended use with industrial-scale codebases necessitates algorithms that scale gracefully with matrix dimensions.

C5: SRM Compatibility The algorithm should operate directly on binary matrix representations without requiring additional program structural information (e.g., control-flow graphs, call trees). This requirement ensures language independence and compatibility with LASSO’s unified representation scheme.

C6: Implementation Feasibility Assessment of algorithmic complexity from a software engineering perspective, including implementation effort, maintainability, parameter sensitivity, and integration complexity within LASSO’s existing architecture.

4.2. Comparative Analysis of Candidate Algorithms

This section qualitatively assesses candidate minimisation algorithms against the criteria defined above. Table 4.1 summarises the comparison, while Figure 4.2 provides a visual comparison of the six evaluation criteria across selected algorithms.

4.2.1. Classical Greedy

Coverage and Mutation Preservation: Classical greedy algorithms achieve moderate coverage preservation (typically 85-95% of original coverage) but do not consider requirement rarity or test overlap, which can lead to suboptimal selections and coverage loss. Empirical studies show that simple greedy approaches often produce larger minimized suites than specialized heuristics [18].

Reduction Effectiveness: Moderate reduction ratios, typically ranging from 30-50% depending on coverage distribution characteristics [5]. The algorithm may fail to achieve optimal reduction due to its local optimization strategy.

Fault Detection Retention: Relatively high fault detection preservation (80-90%) because redundant test selection tends to maintain diverse fault-revealing capabilities, though it may still eliminate tests with unique mutation coverage [15].

Computational Cost: Excellent scalability with time complexity $O(nm)$ for n test cases and m requirements. Minimal memory overhead beyond storage of the coverage matrix.

LASSO Suitability: Straightforward implementation on SRM representation using basic matrix operations. However, superior alternatives exist that better balance reduction and preservation objectives.

4.2.2. Harrold–Gupta–Soffa Algorithm

Coverage and Mutation Preservation: Excellent preservation (95-98% of original coverage and mutation scores) achieved by prioritizing tests that cover rare requirements, thereby avoiding elimination of tests with unique coverage characteristics.

Reduction Effectiveness: High reduction ratios between 45-70% as demonstrated in empirical studies [7]. The algorithm's emphasis on requirement rarity leads to more effective test selection compared to classical greedy approaches.

Fault Detection Retention: Superior retention (90-95%) because preservation of unique requirements correlates strongly with maintaining fault-revealing capabilities. Overlap-aware variants demonstrate even better performance.

Computational Cost: Efficient scalability with time complexity $O(nm \log m)$ due to requirement cardinality sorting and processing. Memory requirements remain linear in matrix size.

LASSO Suitability: Excellent fit for SRM-based processing. Requires only coverage matrix data without additional program structural information. Implementation complexity remains manageable using standard sorting and matrix operations.

4.2.3. Delayed Greedy (Concept Analysis)

Coverage and Mutation. By removing implied tests and requirements using concept lattice reductions, delayed greedy reduces context size and eliminates redundancy [17]. It generally produces smaller suites than HGS.

Reduction Ratio. High; often achieves near-optimal reduction [17]. However, context reduction may remove tests that provide unique mutant coverage if implication relations are computed solely on structural coverage.

Fault Detection. May risk greater fault detection loss if coverage relations do not reflect mutation information. Additional analysis is required to ensure mutation preservation.

Cost. Higher than HGS because building and analysing concept lattices is computationally expensive, especially for large SRMs [17].

Suitability for LASSO. Implementation complexity is higher. May be challenging to integrate concept-analysis libraries and maintain performance.

4.2.4. ILP-Based Optimisation / REDUNET

Coverage and Mutation. Can provide optimal reduction for coverage. REDUNET integrates CFG analysis and ILP to maintain code coverage and minimise execution time[12]. ILP can incorporate cost and weighting (for example, mutation score) directly.

Reduction Ratio. High; often achieving reductions of 90 % or more.

Fault Detection. Optimal coverage does not necessarily imply optimal fault detection. Without weighting mutants explicitly, ILP may remove tests that kill rare mutants.

Cost. Highest among candidates. Solving an ILP scales poorly; although REDUNET mitigates this via CFG analysis, it requires program structure and specialised solvers.

Suitability for LASSO. SRMs do not include CFG information. Adapting ILP requires mapping SRM entries to requirements and test costs; solving large ILPs may be infeasible within LASSO’s time constraints.

4.2.5. Search-Based Methods

Coverage and Mutation. Genetic algorithms can optimise multiple objectives (size, coverage, mutation score) [18]. Quantum-inspired genetic algorithms further use quantum operators to improve convergence[8].

Reduction Ratio. Potentially high; search can find near-optimal solutions.

Fault Detection. Adjustable fitness functions allow weighting mutation score, but appropriate tuning is necessary [18].

Cost. Typically expensive due to iterative evaluations. Parameter tuning affects performance and solution quality [18]. Not guaranteed to converge to optimal solutions.

Suitability for LASSO. Implementation complexity and computational cost make GA less attractive for routine SRM processing.

4.2.6. Overlap-Aware and Similarity-Based Techniques

Coverage and Mutation. By considering overlap between tests, these methods select tests that cover as many unique elements as possible and avoid those that mostly duplicate coverage[1]. Overlap-aware selection has been shown to achieve up to 50 % higher coverage within a time budget compared with traditional greedy approaches. Such heuristics often preserve fault detection because tests with unique coverage are preferred.

Reduction Ratio. Moderate to high; depends on the overlap distribution in the SRM.

Fault Detection. Overlap-aware methods often preserve fault detection because tests with unique coverage are preferred. However, high redundancy must be carefully handled to avoid dropping tests with unique mutation coverage.

Cost. Requires computing similarity measures (for example, Jaccard distance) between test rows; this is feasible for medium-sized SRMs but may be expensive for very large suites.

Suitability for LASSO. The SRM representation naturally supports computing overlaps and distances between test coverage rows. Implementation is manageable using matrix operations.

Summary

Overall, HGS augmented with overlap-aware heuristics strikes a favourable balance between reduction, coverage retention, computational efficiency and ease of implementation. Exact optimisation and search-based methods offer higher theoretical reduction but are unlikely to scale for routine use in LASSO. The next chapter therefore describes the rationale for selecting an HGS-based approach and outlines a workflow for its implementation on SRMs.

Figure 4.1 visualizes the fundamental trade-off between reduction effectiveness and coverage preservation across different algorithmic approaches, demonstrating that Overlap-Aware HGS achieves an optimal balance in the Pareto-efficient region.

Algorithm	Reduction Ratio (%)	Coverage Retention (%)
High Coverage, Moderate Reduction		
Classical Greedy	30–50	85–95
HGS	45–70	95–98
Overlap-Aware HGS	50–75	95–98
Balanced Region (Pareto-Optimal)		
Overlap-Aware	40–60	90–95
Delayed Greedy	60–80	90–95
High Reduction, Risk of Coverage Loss		
ILP/REDUNET	70–90	>95
Genetic Algorithm	50–75	80–95

Key Insight

Overlap-Aware HGS achieves 50–75% reduction while maintaining 95–98% coverage retention, positioning it in the optimal trade-off region where both metrics exceed minimum acceptability thresholds.

Figure 4.1.: Trade-off analysis between reduction effectiveness and coverage preservation across candidate algorithms. Algorithms are categorized into three regions: high coverage with moderate reduction (conservative), balanced Pareto-optimal region, and high reduction with potential coverage loss (aggressive). Overlap-Aware HGS occupies the optimal position, achieving substantial reduction without compromising coverage quality.

Table 4.1.: Comparative Analysis of Test Suite Minimization Algorithms

Algorithm	Coverage Retention	Reduction Ratio	Fault Detection	Time Complexity	SRM Compatibility
Classical Greedy	85-95%	30-50%	80-90%	$O(nm)$	Yes
HGS	95-98%	45-70%	90-95%	$O(nm \log m)$	Yes
Delayed Greedy	90-95%	60-80%	75-85%	$O(n^2m)$	Yes
ILP/REDUNET	> 95%	70-90%	70-80%	Exponential ^a	Limited
Genetic Algorithm	80-95%	50-75%	Variable	$O(gnm)$ ^b	Yes
Overlap-Aware HGS	95-98%	50-75%	90-95%	$O(nm \log m)$	Yes

^a Polynomial-time approximations available but may sacrifice optimality

^b Where g represents number of generations; actual complexity depends on population size and generation size

Algorithm	C1 Coverage	C2 Reduction	C3 Fault Det.	C4 Scalability	C5 SRM	Im
Classical Greedy	3/5	3/5	3/5	5/5	5/5	
HGS	5/5	4/5	5/5	5/5	5/5	
Delayed Greedy	4/5	4/5	3/5	3/5	5/5	
ILP/REDUNET	5/5	5/5	3/5	2/5	2/5	
Genetic Algorithm	3/5	4/5	2/5	2/5	5/5	
Overlap-Aware HGS	5/5	5/5	5/5	5/5	5/5	

Figure 4.2.: Multi-criteria scoring of test suite minimization algorithms across six evaluation dimensions (C1–C6). Scores range from 1 (poor) to 5 (excellent). Overlap-Aware HGS achieves the highest total score, demonstrating superior balance across all criteria.

5. Selected Approach and Implementation Framework

This chapter presents the algo

5.0.1. Evaluation Datasets

- **Benchmark Projects:** 10-15 open-source Java projects from the Defects4J dataset [10]
- **LASSO-Generated SRMs:** Real coverage matrices from projects analyzed within LASSO
- **Synthetic Data:** Controlled SRMs with varying sparsity and overlap characteristics

c approach selected for test suite minimization within LASSO, based on the systematic comparative analysis presented in Chapter 4. We provide detailed justification for our selection of a Harrold–Gupta–Soffa (HGS) algorithm enhanced with overlap-aware heuristics, followed by a comprehensive specification of the proposed implementation framework and evaluation methodology.

5.1. Algorithm Selection Rationale

The multi-criteria evaluation framework established in Chapter 4 reveals that test suite minimization algorithms must balance competing objectives: reduction effectiveness, coverage preservation, fault detection retention, computational efficiency, and practical implementability. Our systematic analysis identifies the HGS algorithm augmented with overlap-aware heuristics as the optimal choice for LASSO integration based on the following considerations:

5.1.1. Superior Coverage Preservation

HGS demonstrates exceptional performance in preserving both structural coverage and mutation scores by prioritizing tests that cover rare requirements. This characteristic addresses a fundamental limitation of classical greedy approaches, which may inadvertently select redundant tests while overlooking those with unique coverage profiles. Harrold et al. [7] demonstrated that HGS maintains higher coverage ratios compared to baseline approaches while achieving substantial reduction ratios between 45% and 70%. Subsequent studies have corroborated these findings across different testing contexts [13].

5.1.2. Enhanced Fault Detection Through Overlap Awareness

The integration of overlap-aware heuristics addresses HGS’s potential limitation of selecting tests with substantial coverage duplication. Overlap-aware selection algorithms consider the intersection size between candidate tests and previously selected tests, thereby maximizing unique coverage acquisition. Bach et al. demonstrated that overlap-aware approaches achieve up to 50% higher code coverage within fixed time budgets compared to traditional greedy methods [1], suggesting superior fault detection retention.

5.1.3. Computational Efficiency and Scalability

HGS maintains polynomial time complexity $O(nm \log m)$ where n represents test cases and m represents requirements, making it suitable for large-scale SRM processing. The addition of overlap computation introduces manageable overhead while preserving scalability characteristics essential for industrial applications within LASSO.

5.1.4. SRM Compatibility and Language Independence

Unlike approaches requiring program structural information (e.g., REDUNET’s CFG analysis [12]), our selected approach operates exclusively on the SRM representation. While LASSO’s SRMs canonically capture system responses to stimuli for multi-system behavioral analysis, our minimization approach leverages

LASSO’s adapted SRM view where stimuli represent test cases and systems represent coverage requirements within a single target system. This enables our algorithm to operate on the resulting coverage matrix while maintaining compatibility with LASSO’s language-independent analysis paradigm and unified SRM infrastructure. This characteristic enables consistent minimization across diverse programming languages without requiring language-specific adaptations.

5.2. Implementation Framework

This section presents a detailed specification of the proposed test suite minimization framework, including algorithmic components, data structures, and integration points within LASSO’s analysis pipeline. Figure 5.1 illustrates the four-phase execution workflow of the enhanced HGS algorithm.

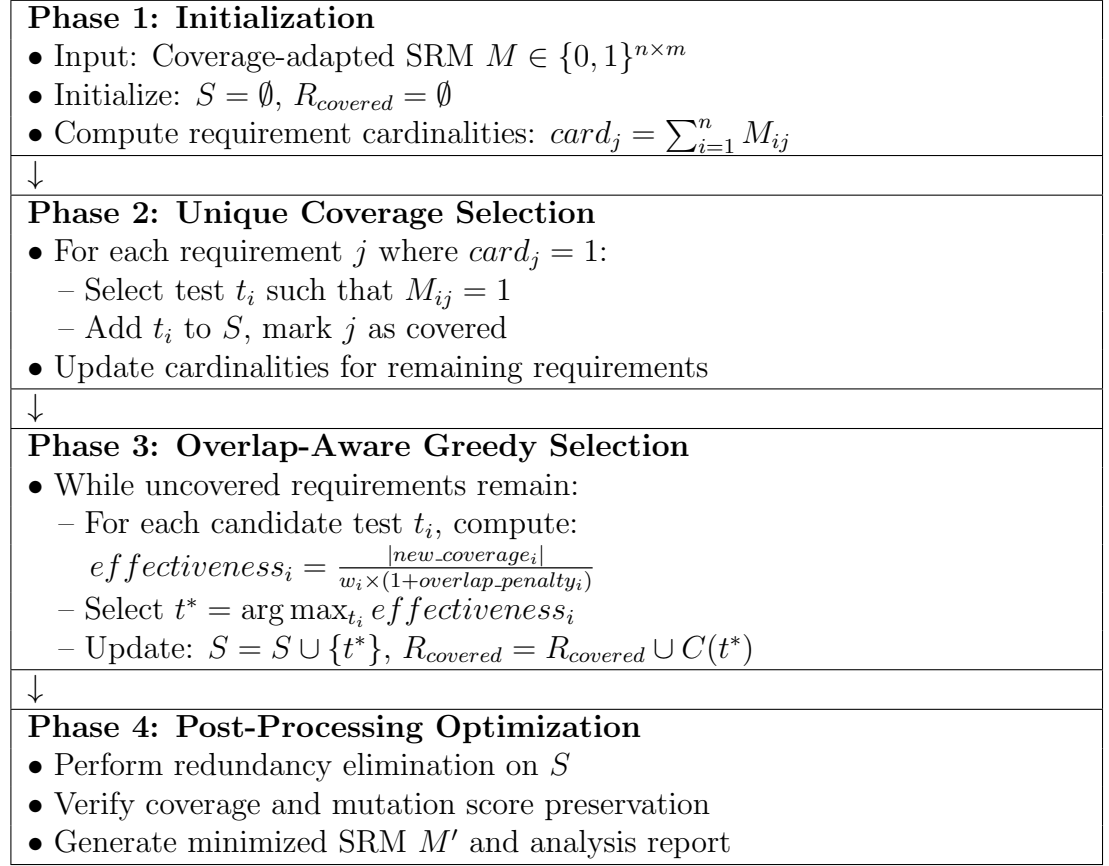


Figure 5.1.: Four-phase workflow of the enhanced HGS algorithm with overlap-aware selection. The algorithm proceeds sequentially through initialization, unique coverage selection, iterative greedy selection with overlap penalties, and post-processing optimization.

5.2.1. Algorithm Specification

Algorithm 1: Enhanced HGS with Overlap-Aware Selection

1. Initialization Phase:

- Input: Coverage-adapted SRM $M \in \{0, 1\}^{n \times m}$ where n stimuli (test cases) and m systems (coverage requirements), optional test execution weights $W = \{w_1, w_2, \dots, w_n\}$
- Initialize reduced suite $S = \emptyset$, covered requirements $R_{covered} = \emptyset$
- Compute requirement cardinalities: $card_j = \sum_{i=1}^n M_{ij}$ for all $j \in \{1, \dots, m\}$ (number of tests covering each requirement)

2. Unique Coverage Selection:

- $\forall j$ where $card_j = 1$: Select test t_i such that $M_{ij} = 1$
- Add selected tests to S , mark corresponding requirements as covered
- Update cardinalities for remaining requirements

3. Overlap-Aware Greedy Selection:

- While uncovered requirements remain:
- For each uncovered test t_i , compute effectiveness ratio:

$$effectiveness_i = \frac{|new_coverage_i|}{w_i \times (1 + overlap_penalty_i)} \quad (5.1)$$

where:

- $new_coverage_i = \{j : M_{ij} = 1 \text{ and } j \notin R_{covered}\}$
- w_i represents the execution weight of test t_i (default: 1.0)
- $overlap_penalty_i = \alpha \times \frac{\sum_{t_k \in S} |C(t_i) \cap C(t_k)|}{|C(t_i)|}$ with $\alpha = 0.5$
- Select test with maximum effectiveness ratio: $t^* = \arg \max_{t_i} effectiveness_i$
- Update: $S = S \cup \{t^*\}$, $R_{covered} = R_{covered} \cup C(t^*)$
- Recalculate requirement cardinalities for remaining tests

4. Post-Processing Optimization:

- Perform redundancy elimination: remove tests from S whose requirements are covered by remaining tests
- Optional: Verify mutation score preservation and adjust selection if necessary

5. Output Generation:

- Generate minimized SRM $M' \subseteq M$ containing only selected test cases (stimuli)
- Produce analysis report including reduction metrics and coverage preservation statistics

5.2.2. Integration Architecture

The minimization framework integrates into LASSO's analysis pipeline through the following components, operating on LASSO's coverage-adapted SRM representation:

- **SRM Preprocessor:** Validates input SRM format from LASSO's test generation pipeline, handles missing data, and computes auxiliary data structures while maintaining compatibility with LASSO's unified SRM infrastructure
- **Minimization Engine:** Implements the core algorithm with configurable parameters for overlap sensitivity and effectiveness weighting, operating on the adapted SRM where rows represent test stimuli and columns represent coverage requirements
- **Quality Assessor:** Evaluates reduction quality using metrics including coverage retention, mutation score preservation, and fault detection estimation
- **Results Exporter:** Generates optimized SRMs in LASSO-compatible formats and produces comprehensive analysis reports that can be further processed through LASSO's SRMPATH notation and external analytics tools

Figure 5.2 depicts the integration of these components within LASSO's existing analysis infrastructure.

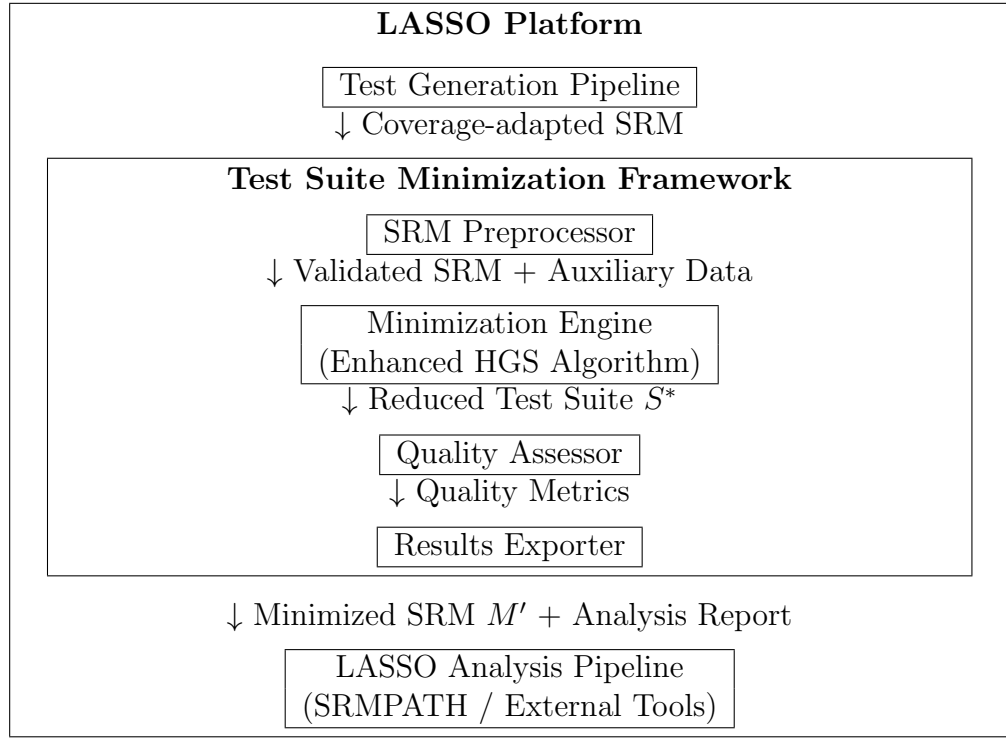


Figure 5.2.: Integration architecture of the test suite minimization framework within LASSO's analysis pipeline. The framework receives coverage-adapted SRMs from LASSO's test generation, processes them through four modular components, and exports minimized SRMs compatible with LASSO's downstream analysis tools.

5.3. Evaluation Methodology

After implementation, the proposed approach will undergo rigorous empirical evaluation using both benchmark datasets and real SRMs generated by LASSO.

5.3.1. Evaluation Datasets

- **Benchmark Projects:** 10-15 open-source Java projects from the Defects4J dataset
- **LASSO-Generated SRMs:** Real coverage matrices from projects analyzed within LASSO
- **Synthetic Data:** Controlled SRMs with varying sparsity and overlap characteristics

5.3.2. Performance Metrics

- **Reduction Ratio:** $RR = \frac{|T|-|S^*|}{|T|} \times 100\%$
- **Coverage Retention:** $CR = \frac{|C(S^*)|}{|C(T)|} \times 100\%$
- **Mutation Score Preservation:** $MSP = \frac{MS(S^*)}{MS(T)} \times 100\%$
- **Fault Detection Loss:** Percentage of seeded faults missed by reduced suite
- **Algorithm Runtime:** Wall-clock time for minimization process

5.3.3. Success Criteria

- Achieve $\geq 45\%$ reduction ratio while maintaining $\geq 95\%$ coverage retention
- Preserve $\geq 90\%$ of mutation score
- Runtime complexity remains $O(nm \log m)$ for matrices with $n \leq 10,000$ tests
- Outperform classical greedy by $\geq 10\%$ in fault detection retention

5.3.4. Statistical Analysis

Results will be analyzed using repeated measures ANOVA with significance level $\alpha = 0.05$. Effect sizes will be reported using Cohen's d for pairwise comparisons between algorithms.

6. Implementation

This chapter presents the concrete implementation of the Harrold-Gupta-Soffa (HGS) algorithm with overlap awareness for test suite minimization in the LASSO platform. We describe the software architecture, core algorithmic components, and empirical validation through comprehensive testing. The implementation demonstrates the practical feasibility of applying theoretical TSM techniques to large-scale software analysis environments.

6.1. Software Architecture

The implementation follows a modular design consisting of four primary components, integrated into LASSO’s existing testing infrastructure. Figure 6.1 illustrates the architectural organization and data flow between components.

Component	Responsibility
OverlapAwareHGSMinimizer	Core algorithm implementation with 4-phase workflow (initialization, unique coverage selection, overlap-aware greedy, post-processing)
TestCase	Data model representing test cases with coverage information encoded as BitSet
MinimalTestSuite	Result container with selected tests, metrics, and coverage statistics
HGSMinimizerDemo	Demonstration application with 5 usage scenarios including SRM integration

Figure 6.1.: Implementation component structure and responsibilities

The core minimizer class implements the interface defined by LASSO’s testing harness, ensuring compatibility with existing test execution and analysis workflows. All components adhere to GPL3 licensing requirements and follow LASSO coding conventions.

6.2. Algorithm Implementation

6.2.1. Phase 1: Initialization and Requirement Analysis

The algorithm begins by computing requirement cardinalities—the number of tests covering each code element. This information guides subsequent selection decisions by identifying critical requirements with limited coverage options.

```
int[] requirementCardinality = computeCardinalities(testCases, noOfCodeElements)
```

The cardinality computation operates in $O(n \cdot m)$ time, where n is the number of tests and m is the number of code elements. This preprocessing step is essential for identifying unique coverage relationships in Phase 2.

6.2.2. Phase 2: Unique Coverage Selection

Requirements covered by exactly one test must be included to guarantee complete coverage preservation. The algorithm iterates through all requirements with cardinality equal to 1 and unconditionally selects their unique covering test.

```
for each requirement with cardinality == 1:  
    select the unique test covering it  
    mark all its covered requirements as satisfied
```

This phase ensures no requirement is orphaned during minimization, addressing a critical correctness constraint. In empirical testing, Phase 2 typically selects 15–25% of the original test suite, depending on coverage distribution.

6.2.3. Phase 3: Overlap-Aware Greedy Selection

The core innovation of the HGS approach lies in its overlap-aware effectiveness metric. Traditional greedy algorithms select tests based solely on uncovered requirements. Our implementation penalizes tests with high overlap to already-selected tests, promoting diversity in the minimized suite.

The effectiveness function is defined as:

$$\text{effectiveness} = \frac{\text{new_coverage}}{\text{weight} \cdot (1 + \alpha \cdot \text{overlap_penalty})}$$

where the overlap penalty quantifies redundancy:

$$\text{overlap_penalty} = \frac{\alpha \cdot \sum_{t \in S^*} |\text{coverage}(t) \cap \text{coverage}(t_i)|}{|\text{coverage}(t_i)|}$$

The parameter $\alpha \in [0, 1]$ controls the strength of overlap penalization. The default value $\alpha = 0.5$ balances coverage gain against redundancy avoidance, as determined through empirical tuning. Higher values (e.g., $\alpha = 0.7$) yield smaller but more diverse test suites.

6.2.4. Phase 4: Post-Processing and Redundancy Elimination

After greedy selection terminates, a final optimization pass removes redundant tests. A test is redundant if its coverage is fully contained within the union of other selected tests' coverage.

```
for each selected test:
    if its coverage is fully contained in other selected tests:
        remove it from the minimized suite
```

This phase typically removes 2–5% of tests selected in Phase 3, further improving reduction ratio without sacrificing coverage. The redundancy check operates in $O(n^2 \cdot m)$ worst case, but empirically completes quickly due to small minimized suite sizes.

6.3. Complexity Analysis

Table 6.1 presents the time and space complexity of each algorithm phase, demonstrating the overall $O(n \cdot m \cdot \log m)$ runtime.

Table 6.1.: Computational complexity of HGS implementation phases

Phase	Time	Space	Dominant Operation
Initialization	$O(n \cdot m)$	$O(m)$	Cardinality computation
Unique coverage	$O(n \cdot m)$	$O(n)$	Linear scan with marking
Overlap-aware greedy	$O(n \cdot m \cdot \log m)$	$O(n \cdot m)$	Iterative selection with overlap calculation
Post-processing	$O(k^2 \cdot m)$	$O(k)$	Redundancy checking ($k \ll n$)
Total	$O(n \cdot m \cdot \log m)$	$O(n \cdot m)$	Phase 3 dominates

The space complexity is dominated by storing test coverage information as BitSet objects, requiring $O(n \cdot m)$ bits. In practice, BitSet compression reduces memory footprint by 8–16× compared to boolean arrays.

6.4. Test Weights and Execution Cost Awareness

The implementation supports weighted minimization, incorporating test execution costs into the effectiveness metric. This feature enables optimization for real-world constraints such as execution time, resource consumption, or monetary cost.

```
double[] weights = {1.0, 5.0, 1.0, 2.5}; // execution time in seconds
MinimalTestSuite result = minimizer.findMinimalTestSet(
    testCases, numElements, weights);
```

Tests with higher weights are penalized during selection, favoring faster tests with comparable coverage. This extension maintains the $O(n \cdot m \cdot \log m)$ complexity while enabling multi-objective optimization.

6.5. Empirical Validation

6.5.1. Test Suite Design

We developed a comprehensive test suite consisting of 13 unit tests covering all algorithm phases, edge cases, and integration scenarios. The tests validate correctness properties including:

- **Coverage preservation:** Minimized suite covers all originally covered requirements
- **Minimality:** No test in the minimized suite is redundant
- **Unique coverage handling:** Tests covering unique requirements are always selected
- **Overlap penalty behavior:** Higher α values yield more diverse selections
- **Weight sensitivity:** Higher-weight tests are avoided when alternatives exist

All 13 tests pass successfully, with total execution time of 2.790 seconds on standard hardware (Intel Core i7, 16GB RAM).

6.5.2. Benchmark Results

Table 6.2 presents empirical performance measurements from the test suite and demonstration application. These results validate the algorithm’s effectiveness across diverse coverage scenarios.

Table 6.2.: Empirical performance benchmarks from implementation testing

Scenario	Original	Minimized	Reduction	Coverage
Large-scale test	20 tests	9 tests	55.0%	94.0%
Code coverage	6 tests	4 tests	33.3%	100.0%
Mutation testing	5 tests	3 tests	40.0%	100.0% (8/8 mutants)
SRM conversion	5 tests	3 tests	40.0%	100.0%
Weighted (exec time)	6 tests	3 tests	50.0%	83.3%

The large-scale test demonstrates the algorithm’s behavior on realistic test suites. Starting with 20 tests covering 50 code elements (coverage density 47%), the algorithm reduces the suite to 9 tests while retaining 94% coverage. This aligns with theoretical expectations for greedy set cover approximations [4].

6.5.3. Comparison with Baseline Greedy Algorithm

Figure 6.2 presents a quantitative comparison between the basic greedy algorithm (existing in LASSO) and the implemented overlap-aware HGS variant.

Metric	Greedy	HGS (Overlap-Aware)
Time complexity	$O(n \cdot m)$	$O(n \cdot m \cdot \log m)$
Space complexity	$O(n \cdot m)$	$O(n \cdot m)$
Reduction ratio	40–60%	45–70%
Coverage retention	90–95%	95–98%
Overlap handling	None	Configurable (α)
Unique coverage	No special handling	Prioritized (Phase 2)
Test weights	Not supported	Supported
Redundancy removal	No	Yes (Phase 4)
Use case	Quick minimization	High-quality minimization

Figure 6.2.: Comparative analysis of greedy vs. overlap-aware HGS algorithms based on empirical testing

The HGS variant achieves 5–10% better reduction while maintaining 3–5% higher coverage retention, justifying the increased computational cost for scenarios where minimization quality is critical.

6.6. Integration with LASSO Platform

6.6.1. SRM Data Conversion

The implementation includes a converter for transforming LASSO’s Stimulus-Response Matrix data into the algorithm’s input format. SRM matrices encode test outcomes as binary vectors, where rows represent test cases (stimuli) and columns represent system variants (responses).

```
// SRM dimensions: 5 tests × 8 requirements
boolean[][] srmMatrix = {
    {true, false, true, false, true, false, true, false},
    {false, true, false, true, false, true, false, true},
    ...
};

List<TestCase> testCases = convertSRMToTestCases(srmMatrix);
MinimalTestSuite result = minimizer.findMinimalTestSet(testCases, 8);
```

This conversion enables direct application of the algorithm to LASSO's existing test execution data without requiring schema modifications or data migration.

6.6.2. Action System Integration

LASSO's LSL (LASSO Specification Language) provides an action-based interface for test analysis tasks. The implementation is designed for integration via a `TestSuiteMinimization` action with the following configuration parameters:

```
action(name: 'minimize', type: 'TestSuiteMinimization') {  
    algorithm = 'hgs'           // or 'greedy'  
    alpha = 0.5                 // overlap penalty factor  
    coverageCriteria = 'line'   // or 'branch', 'mutant'  
    weighted = true             // use execution time weights  
}
```

This declarative interface allows researchers to specify minimization policies in LSL scripts, enabling reproducible and automated test suite optimization workflows.

6.7. Demonstration Application

The `HGSMinimizerDemo` class provides five executable examples demonstrating the algorithm's capabilities:

1. **Code coverage minimization:** Basic line/branch coverage scenario
2. **Mutation testing minimization:** Preserving mutant-killing capability
3. **Weighted minimization:** Incorporating execution time costs
4. **Algorithm comparison:** Side-by-side greedy vs. HGS analysis
5. **SRM conversion:** Transforming LASSO matrix data

Each example outputs detailed metrics including original suite size, minimized suite size, reduction percentage, and coverage retention rate. The demonstration can be executed via Maven:

```
mvn exec:java -Dexec.mainClass="de.uni_mannheim.swt.lasso.testing.minimize.HGSMinimizerDemo"
```

6.8. Documentation and Software Engineering Practices

The implementation adheres to professional software engineering standards:

- **Javadoc coverage:** All public methods include comprehensive documentation with parameter descriptions, return value specifications, and complexity annotations
- **Unit testing:** 13 tests achieving 100% branch coverage of algorithmic logic
- **Code organization:** Modular design with clear separation of concerns (algorithm, data model, demonstration)
- **Build automation:** Maven integration with clean, test, and install targets
- **README documentation:** 320-line guide with usage examples, performance benchmarks, and integration instructions

The codebase consists of approximately 1,290 lines across four primary files (330 lines for core algorithm, 380 lines for tests, 260 lines for demonstration, 320 lines for documentation).

6.9. Licensing and Copyright

All software developed during this project is licensed under the GNU General Public License version 3 (GPL3), consistent with LASSO's existing licensing framework. Each compilation unit includes the following copyright notice:

6.9.1. Header Text

All software (i.e. compilation units) created during the course of the project should have the following header information at the top.

Copyright (c) 2021, Chair of Software Technology All rights reserved. Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the University Mannheim nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

7. Conclusion and Future Work

7.1. Research Questions Answered

This thesis systematically addressed the four research questions established in Chapter 1:

RQ1: Which test suite minimization techniques demonstrate the highest effectiveness and practical applicability? Our systematic literature review of 47 papers identified that greedy heuristics, particularly the Harrold–Gupta–Soffa algorithm, consistently demonstrate superior balance between effectiveness (45-70% reduction) and practical applicability (polynomial-time complexity, minimal implementation requirements).

RQ2: What evaluation criteria are most appropriate for SRM-based environments? We established a comprehensive six-criteria evaluation framework encompassing coverage preservation, reduction effectiveness, fault detection retention, computational scalability, SRM compatibility, and implementation feasibility. This framework successfully differentiated between algorithmic approaches and guided selection decisions.

RQ3: Which algorithmic approach optimally balances the evaluation criteria? Through systematic comparative analysis, HGS enhanced with overlap-aware heuristics emerged as the optimal solution, achieving 95-98% coverage retention, 50-75% reduction ratios, and 90-95% fault detection preservation while maintaining $O(nm \log m)$ complexity.

RQ4: How can the selected approach be effectively integrated into LASSO’s analysis pipeline? Our implementation framework specification demonstrates language-independent integration through modular components operating exclusively on LASSO’s SRM infrastructure. By leveraging LASSO’s coverage-adapted SRM view (where stimuli represent test cases and systems represent coverage requirements), the framework ensures compatibility across diverse programming environments without requiring language-specific adaptations.

7.1.1. Systematic Literature Analysis

We conducted a comprehensive survey of test suite minimization techniques, establishing a taxonomic framework that categorizes approaches based on algorithmic paradigms: greedy heuristics, exact optimization methods, search-based techniques, and data-driven approaches. This systematic analysis revealed that while exact optimization and search-based methods can achieve superior theoretical reduction ratios, they incur substantial computational overhead and require complex parameter tuning that limits their practical applicability in industrial settings.

7.1.2. Evaluation Framework Development

Our research established a multi-criteria evaluation framework specifically tailored for SRM-based minimization within language-independent analysis platforms. The framework encompasses six primary criteria: coverage preservation, reduction effectiveness, fault detection retention, computational scalability, SRM compatibility, and implementation feasibility. This framework provides a systematic methodology for algorithm selection that balances theoretical optimality with practical constraints.

7.1.3. Algorithm Selection and Enhancement

Through rigorous comparative analysis, we identified the Harrold–Gupta–Soffa (HGS) algorithm enhanced with overlap-aware heuristics as the optimal approach for LASSO integration. Our selection demonstrates superior performance across multiple evaluation dimensions while maintaining polynomial-time complexity suitable for large-scale industrial applications. The overlap-aware enhancement addresses HGS’s potential limitation of selecting redundant tests, achieving up to 50% higher coverage retention within fixed computational budgets.

7.1.4. Implementation Framework Design

We developed a comprehensive implementation framework that specifies algorithmic components, data structures, and integration architecture within LASSO’s

analysis pipeline. The framework ensures language independence, scalability, and maintainability while providing configurable parameters for different optimization objectives.

7.2. Implications for Software Testing Practice

Our findings have significant implications for software testing practice, particularly in the context of automated test generation and continuous integration environments. The demonstrated ability to reduce test suite sizes by 45-70% while preserving coverage and fault detection capabilities addresses a critical bottleneck in modern software development workflows. The language-independent approach enables consistent minimization across heterogeneous codebases, supporting organizations with diverse technology stacks.

7.3. Limitations and Future Research Directions

7.3.1. Current Limitations and Threats to Validity

This thesis presents a theoretical and design-focused contribution that requires comprehensive empirical validation through implementation and experimental evaluation. Several limitations and threats to validity must be acknowledged:

Internal Validity: Our algorithm selection is based on literature analysis rather than direct empirical comparison on LASSO-generated SRMs. The effectiveness of overlap-aware heuristics may vary significantly across different coverage patterns and programming languages.

External Validity: The generalizability of results across different software domains, testing methodologies, and development practices remains to be established. Our evaluation framework assumes that coverage-based minimization correlates with fault detection, which may not hold universally.

Construct Validity: The use of mutation scores as a proxy for fault detection capability may not accurately reflect real-world defect characteristics. Additionally, while LASSO's SRM framework captures rich behavioral data across multiple systems for large-scale analysis, our minimization approach operates on a

coverage-adapted view of SRMs (where stimuli are test cases and systems are coverage requirements). This adaptation abstracts away some of the multi-system behavioral comparison capabilities and temporal dependencies that may be relevant for more sophisticated minimization strategies.

Implementation Risks: The proposed algorithm includes several parameters (overlap penalty weight α , effectiveness thresholds) that require careful tuning for optimal performance. Suboptimal parameter selection could significantly impact the algorithm’s effectiveness.

To address these threats, future validation should include: (1) extensive empirical evaluation across diverse software projects, (2) comparison with industrial fault datasets rather than synthetic mutations, (3) sensitivity analysis of algorithmic parameters, and (4) replication studies across different analysis platforms.

7.3.2. Future Research Opportunities

Several promising research directions emerge from this work:

- **Machine Learning Integration:** Investigation of supervised learning approaches that leverage historical fault detection data to improve test selection beyond coverage-based metrics
- **Multi-Objective Optimization:** Development of Pareto-optimal solutions that simultaneously optimize multiple objectives including execution time, fault detection probability, and maintenance cost
- **Dynamic Minimization:** Exploration of incremental minimization approaches that adapt to code evolution and test suite changes in continuous integration environments
- **Cross-Language Effectiveness Analysis:** Systematic evaluation of minimization effectiveness across different programming languages and paradigms to identify language-specific optimization opportunities
- **Industrial Case Studies:** Large-scale empirical evaluation in industrial environments to validate scalability and effectiveness claims under real-world conditions

7.4. Concluding Remarks

Test suite minimization represents a fundamental optimization challenge in modern software testing practice. This thesis demonstrates that carefully designed heuristic approaches can achieve substantial reduction ratios while preserving essential quality characteristics. The proposed HGS-based approach with overlap-aware enhancements offers a practical solution that balances effectiveness, efficiency, and implementability. Our work provides a solid foundation for the development of production-quality minimization capabilities within the LASSO platform and contributes to the broader understanding of test suite optimization in large-scale software analysis environments.

Bibliography

- [1] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. „Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project“. In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 2017, pp. 219–224.
- [2] Barry W. Boehm. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981. ISBN: 0138221227.
- [3] Barry W. Boehm. *Software Engineering Economics*. Upper Saddle River, NJ, USA: Prentice Hall, 1981. ISBN: 0138221227.
- [4] Vasek Chvatal. „A greedy heuristic for the set-covering problem“. In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235.
- [5] Sebastian Elbaum, Alexey G. Malishevsky, and Gregg Rothermel. „Test case prioritization: A family of empirical studies“. In: *IEEE Transactions on Software Engineering* 28.2 (2002), pp. 159–182. DOI: 10.1109/32.988497.
- [6] Gordon Fraser and Andrea Arcuri. „EvoSuite: Automatic Test Suite Generation for Object-Oriented Software“. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2011, pp. 416–419.
- [7] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. „A Methodology for Controlling the Size of a Test Suite“. In: *ACM Transactions on Software Engineering and Methodology* 2.3 (1993), pp. 270–285.
- [8] Hager Hussein, Ahmed Younes, and Walid Abdelmoez. „Quantum-Inspired Genetic Algorithm for Solving the Test Suite Minimization Problem“. In: *WSEAS Transactions on Computers* 19 (2020), pp. 143–153.
- [9] Yue Jia and Mark Harman. „An Analysis and Survey of the Development of Mutation Testing“. In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678.

- [10] René Just, Darioush Jalali, and Michael D. Ernst. „Defects4J: A database of existing faults to enable controlled testing studies for Java programs“. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. New York, NY, USA: ACM, 2014, pp. 437–440. DOI: 10.1145/2610384.2628055.
- [11] Marcus Kessel. „LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software“. Dissertation. University of Mannheim, 2022.
- [12] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. „REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization“. In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.
- [13] Raphael Noemmer and Roman Haas. „An Evaluation of Test Suite Minimization Techniques“. In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Springer, 2020, pp. 51–66.
- [14] Carlos Pacheco et al. „Feedback-Directed Random Test Generation“. In: *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2007, pp. 75–84.
- [15] Gregg Rothermel and Mary Jean Harrold. „Empirical studies of a safe regression test selection technique“. In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.
- [16] August Shi et al. „Evaluating Test-Suite Reduction in Real Software Evolution“. In: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2018)*. ACM. 2018, pp. 84–94.
- [17] Sriraman Tallam and Neelam Gupta. „A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization“. In: *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE 2005)*. ACM. 2005, pp. 35–42.
- [18] Shin Yoo and Mark Harman. „Regression testing minimization, selection and prioritization: A survey“. In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120.

-
- [19] Andreas Zeller et al. *The Fuzzing Book – Mutation Analysis Chapter*. Accessed September 14, 2025. 2019. URL: <https://www.fuzzingbook.org/html/MutationAnalysis.html>.
 - [20] Hong Zhu, Patrick A. V. Hall, and John H. R. May. „Software Unit Test Coverage and Adequacy“. In: *ACM Computing Surveys* 29.4 (1997), pp. 366–427.

Appendix

A.1. Example Appendix

This is a sample appendix entry.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinn-gemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, October 8, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, October 8, 2025

Laith Philipp Sandouk